HƯỚNG DẪN PYTHON · NHÓM 15 · 5 SINH VIÊN

Ăn hết thức ăn rồi tới G

Mã hoàn chỉnh, chỉ dùng Python 3.10 trở lên và thư viện chuẩn. Trước hết chạy ví dụ, sau đó mới đọc lõi và thay bản đồ.

1. Mở đúng thư mục

Giải nén toàn bộ ZIP. Trong VS Code: File → Open Folder, mở thư mục có trực tiếp `chay_mau.py`. Chọn Terminal → New Terminal. Không cần `cd python`, không chạy bên trong ZIP.

```
python kiem_tra_moi_truong.py
python chay_mau.py
python ucs_astar_toi_gian.py
python -m unittest -v test_mau test_nhom_vi_du
```

Nếu báo không thấy tệp, tìm thư mục bằng PowerShell:

```
Get-ChildItem -Recurse -Filter chay_mau.py
```

Mở đúng thư mục hoặc dùng đường dẫn đầy đủ trong dấu ngoặc kép. Chọn Python: Select Interpreter nếu có nhiều môi trường. Nếu Terminal có >>>, thoát chế độ Python rồi nhập lệnh ở Terminal.

2. Sửa vùng dữ liệu

Trong `chay_mau.py`, sửa các biến đầu; không cần sửa lõi:

```
ROWS = ['%%%%%%%', '%P.F.G%', '%%%%%%%']
ALGORITHM = 'astar'  # ucs / astar
HEURISTIC = 'food'   # food / via / zero
ORDER = 'RDLU'
```

Các hàng cùng độ dài; đúng một P/G, F là thức ăn. Ví dụ này có đường RRRR giá 4, ăn F ở bước 2, kết thúc bước 4. Không dùng ví dụ làm quen thay N1/N2.

Các tệp quan trọng

`maze.py`: luật/mask/heuristic; `search.py`: tìm kiếm đầy đủ; `ucs_astar_toi_gian.py`: bản ngắn; `run.py`: cấu hình/xuất tệp; `data_cases.py`: các bản đồ chung.

2. Đọc trạng thái và cập nhật thức ăn

Từ tập còn lại đến mask

F đánh thứ tự hàng rồi cột. Với a,b,c, mask 7 có nhị phân 111. Bit 1 chỉ chưa ăn. Ăn b xóa bit thứ 1:

```
new_mask = mask & ~(1 << 1)  # 111 -> 101, so 7 -> 5
```

`&`: AND; `~`: đảo bit; `1 « i`: bit thứ i. Không dùng XOR vì đi lại một ô sẽ bật lại thức ăn.

Sinh trạng thái con

```
q = (r + dr, c + dc)
if q in maze.cells:
    new_mask = mask & ~food_bits.get(q, 0)
    child = (*q, new_mask)
```

Kiểm ô trong cells loại cả tường và ngoài biên. Nếu q không phải F, bit cần xóa là 0 nên giữ mask. Bản đồ không bị sửa; các F đã ăn được lưu bằng mask.

Ví dụ quay lại ô cũ

`FPG`: bắt đầu ở P với mask 1; trái ăn F thành mask 0, quay lại P mask 0, rồi phải tới G. LRR giá 3. Hai lần đứng P là hai state khác nhau. Khóa của từ điển là `(r,c,mask)`, không phải chỉ `(r,c)`.

```
finished = ((r, c) == maze.goal and mask == 0)
```

Đọc hai heuristic

food lấy xa nhất tới F còn bit 1, tập rỗng trả 0. via xét nghĩa vụ qua từng F rồi đến G, đồng thời có hạng tử khoảng cách tới G. Không cộng mọi khoảng cách tới F. Khi ăn viên cuối, food có thể0 dù còn phải đi tới G.

Đọc JSON

`states` lưu toàn bộ bộ ba; `path` chỉ vị trí; `actions` là các hướng. F và bit được đối chiếu qua danh sách `foods`. Hình dấu * không thể hiện đầy đủ số lần quay lại; cần đọc states/actions.

3. Đọc vòng lặp UCS và A*

Khởi tạo và lấy phần tử

```
start = maze.initial()
best_g = {start: 0}; parent = {start: None}
heap = [(H(start), next(ids), 0, start)]
f, _, g, s = heappop(heap)
if g != best_g.get(s):
    continue
```

Một bản ghi là $(f, id, g, state)$; cùng f thì ticket giữ FIFO. H bằng 0 cho UCS. Bỏ bản ghi cũ trước khi kiểm đích. Sau kiểm đích mới kiểm ngân sách; nhờ vậy đầu đã hoàn tất vẫn thành công khi ngân sách 0.

Chỉ cập nhật khi giá rẻ hơn

```
new_g = g + 1
if new_g < best_g.get(child, float('inf')):
    best_g[child] = new_g
    parent[child] = (s, action)
    heappush(heap, (new_g + H(child), next(ids), new_g, child))
```

Không đổi cha khi bằng giá. Bản ghi cũ không cần xóa ngay khỏi heap; sẽ bị bỏ khi lấy. Không khóa vĩnh viễn một vị trí. Khác biệt UCS/A* là H, không phải bỏ cập nhật g.

Dựng đường

Từ state đích, đi theo parent đến đầu, thu actions ngược rồi đảo danh sách. Cost bằng số hành động; số states lớn hơn số bước đúng1. Các trạng thái liên tiếp cập nhật mask theo ô vừa tới; cuối phải có mask 0 và vị trí G.

Ba loại dừng

found: có đường; no_path: hết biên mà chưa hoàn tất; limit: chưa đủ ngân sách. Hai trường hợp sau trả cost/path/states=None, không ghi0 như lời giải. Mặc định 200000 lần mở rộng đủ cho dữ liệu bài; thay ngân sách để thử giới hạn nhưng không dùng kết quả đó thay batch chuẩn.

Mẹo đọc mã

Đặt breakpoint ở heappop hoặc dòng xóa bit, chạy Debug trong VS Code. F10 đi từng dòng, F11 đi vào hàm. Quan sát state, best_g, parent, heap và mask; dự đoán trước khi tiếp tục. Không cần bật step cùng debugger.

4. Thực nghiệm và xem từng bước

```
python run.py --batch --tag bat_buoc
python run.py --case M0 --algo ucs --tag M0_UCS
python run.py --case M0 --heuristic food --tag M0_Afood
python run.py --case M0 --heuristic via --tag M0_Avia
python run.py --case H --algo ucs --trace 6 --tag tay_H_UCS
python run.py --case H --heuristic via --trace 6 --tag tay_H_Avia
python run.py --case H --heuristic via --step --tag demo_step
```

Batch gồm 24 dòng chung. Trace 6 chỉ ghi sáu lần lấy, không dừng tìm kiếm ở bước 6. Step dừng trực tiếp trong vòng lặp thật; nhấn Enter ở Terminal, Ctrl+C dừng hẳn. Không dùng step cho batch.

Đọc một dòng trace

`frontier_before`: trước lấy; state: phần tử lấy; g,h,f: điểm; updates: con thêm/cập nhật; frontier_after: sau xử lý. Biên đã sắp theo thứ tự lấy, không phải mảng heap gốc. Trường stale chỉ bản ghi cũ. Đừng lấy trạng thái sau để gọi là trước.

Tệp đầu ra

output/tag.json có dữ liệu, đường và trace; tag.csv tóm tắt mỗi cấu hình; tag_trace.csv ghi các sự kiện khi bật trace. Không sửa JSON bằng tay. Tên tag chỉ dùng chữ không dấu, số, gạch dưới hoặc gạch nối.

Tạo dữ liệu riêng

```
python tao_du_lieu.py --seed 1526
python run.py --data du_lieu_nhom.json --batch --tag tu_tao
```

N1 có ba F, một F ngõ cụt nên phải quay lại; hai vùng nối qua cửa. N2 chuyển G về phía P rồi đóng cửa, giữ F ở vùng bên kia. Phải đọc hình, giải thích vì sao tới G vẫn chưa đủ. JSON có cases chứa id và rows. Không thêm khoảng trắng cuối hàng.

5. Kiểm thử riêng và báo cáo

Viết một test có kỳ vọng tính tay

```
import unittest
from maze import parse_maze
from search import solve
class TestNhom(unittest.TestCase):
    def test_hanh_lang_tu_tinh(self):
        m = parse_maze(['PFG'])
        r = solve(m, h='via')
        self.assertEqual(r['cost'], 2)
        self.assertEqual(r['actions'], ['R', 'R'])
if __name__ == '__main__':
    unittest.main()
```

Ví dụ cho cách viết, không tính vào mười test riêng. Thay dữ liệu và tự suy kỳ vọng; kiểm đường/mask, không chỉ cost. Năm người mỗi người hai test; ít nhất ba kỳ vọng tính tay.

```
python -m unittest -v test_nhom
python chay_kiem_thu.py
```

Lệnh thứ hai lưu log thật vào output/ket_qua_test.txt. Nhóm tự tạo test_nhom.py; việc tệp ấy chưa có không phải mã thuật toán bị bỏ trống. Test mẫu đã đạt không có nghĩa đã làm test riêng.

Đối chiếu đường

Đầu đúng P, cuối đúngG, mọi bước kề và đi được. Mọi F xuất hiện trên đường, mask không bật thêm bit, cuối mask 0. Tọa độ lặp lại có thể hợp lệ. Nếu chưa có đường thì không cố tính length0.

Điền mẫu báo cáo

Chọn Word hoặc LaTeX trong MauBaoCao; điền năm thành viên, thuật toán bằng lời, số liệu và nhận xét thật. PDF tính tay riêng. Báo cáo 6–8 trang chính và 6–8 slide. Không dán toàn bộ code hoặc trace dài vào phần chính.

Nhận xét phải có bằng chứng

So các cặp cùng bản đồ, thứ tự, ngân sách. Nêu điều giả thuyết sai sau khi chạy. Không suy thời gian luôn tốt hơn từ cận lớn hơn; nếu đo, tắt step/trace và lấy trung vị ba lần trên cùng máy.

6. Đóng gói, tái chạy và bảo vệ

Hoàn thiện tệp trước khi đóng gói

Đổi README_NOP_MAU.md thành README_NOP.md, DongGop_MAU.md thành DongGop.md rồi điền. Có BaoCao.pdf, TinhTay.pdf, ThuyetTrinh.pptx, test_nhom.py, dữ liệu và output trước khi chạy:

```
python dong_goi.py
```

Công cụ chỉ kiểm thành phần và số dòng 24/12; không chứng nhận nội dung đúng, không xác nhận test là do nhóm tự viết. Không tạo tệp rỗng giả để vượt kiểm tra. Không nộp venv/Anaconda hoặc cache.

Tái chạy từ ZIP sạch

Giải nén vào thư mục mới, chạy theo README trên máy thành viên khác. Kiểm số liệu PDF khớp JSON/CSV, code không cần đường dẫn của máy cũ. Thời gian chạy không cần giống đến từng số thập phân.

Tập bảo vệ

Khung 18 phút: 5 phút trình bày, 3 phút demo, 10 phút hỏi năm người (2 phút/người). Mỗi người chuẩn bị giải thích trạng thái, một lần lấy heap, test phụ trách và thay đổi F/G. Trong demo phải dự đoán trước rồi chạy.

Lỗi thường gặp và cách tự kiểm

Không mở được file: kiểm thư mục. Tập F rỗng bị lỗi max: dùng giá trị mặc định. Ăn rồi F trở lại: kiểm XOR. TớiG quá sớm: kiểm đủmask 0. Chỉ tính vị trí: kiểm khóa từ điển. Mask sai sau sửa dữ liệu: dùng lại bộ đọc và ánh xạ F mới.

> Mục tiêu là hiểu mô hình và kiểm chứng. Dùng mã mẫu được cho phép, nhưng cả năm thành viên phải giải thích vì sao vị trí đơn lẻ chưa đủ và vì sao h_food bằng 0 chưa chắc đã hoàn tất.